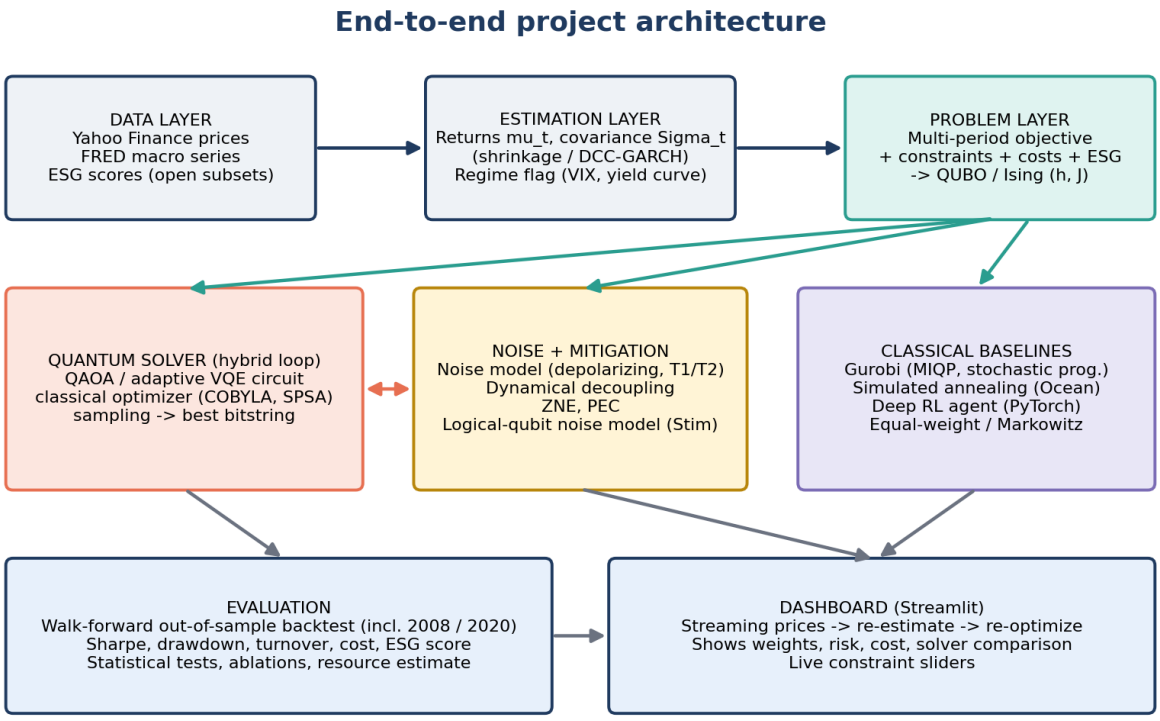
Quantum Computing for Finance

A zero-to-project learning guide for the Fault-Tolerant Quantum Portfolio Optimizer

Everything you need to understand, build and defend the project in 4-5 days: quantum basics, optimization on quantum computers (QUBO, Ising, QAOA, VQE), portfolio formulation, noise and error mitigation, surface codes and resource estimation, honest quantum-advantage analysis, statistics, tech stack and a day-by-day plan.



The whole project on one page. This guide teaches you each box.

0. Read this first

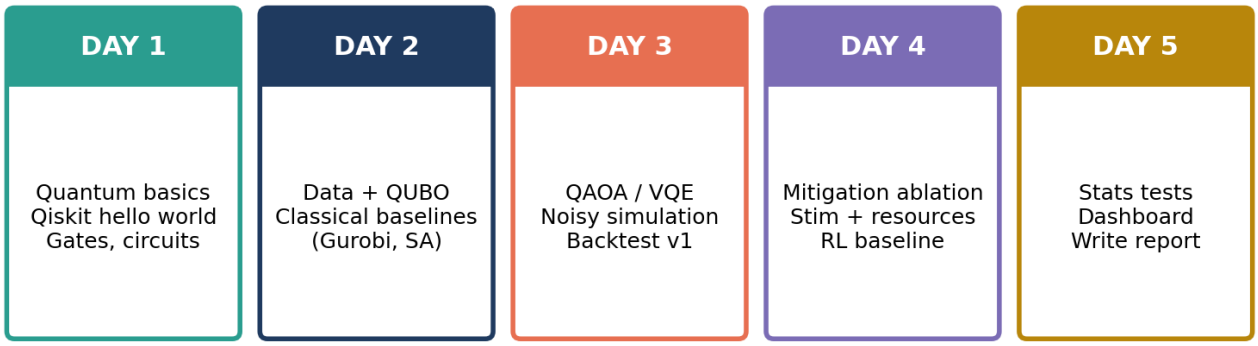
0.1 The honest reality check (important)

Your problem statement asks you to *prove or rigorously bound* a quantum advantage. Be aware of three facts before you start, because your evaluators (and your own report) will be stronger if you state them openly:

- **No proven quantum advantage exists for portfolio optimization on QUBO today.** QAOA and VQE are heuristics. There is no theorem saying they beat Gurobi on realistic portfolio problems. Real, provable speedups in finance are narrow: for example **quantum amplitude estimation** gives a quadratic speedup for Monte Carlo risk estimation (Section 8).
- **You cannot run real fault-tolerant quantum computers in 5 days.** Everything will be *simulated*. Statevector simulation of n qubits costs memory 2^n, so you can handle roughly 20-28 qubits comfortably. Noisy density-matrix simulation tops out around 12-14 qubits.
- **Stim cannot simulate QAOA directly.** Stim simulates only Clifford circuits (very fast, thousands of qubits), but QAOA uses non-Clifford rotation gates. So the standard trick is a *two-stage model*: use Stim to measure the logical error rate of a surface code, then inject that error rate as an effective noise model into a small QAOA simulation (Section 7.4).

How to frame your contribution so it is defensible
Claim **(1)** a complete, reproducible hybrid pipeline; **(2)** an empirical comparison against strong classical baselines with proper statistics; **(3)** a rigorous *resource estimate* showing what hardware would be needed; **(4)** a theoretical analysis giving *bounds* (approximation ratio for QAOA at depth p, sample complexity of amplitude estimation, noise-induced limits). If quantum does not win, that is a valid, publishable negative result as long as you test it correctly.

0.2 How to use this guide



Suggested 5-day schedule. Chapters 1-3 on Day 1-2, Chapters 4-7 on Day 3-4, Chapters 8-10 on Day 5.

Chapter	What you learn	Needed for
1. Quantum basics	Qubits, superposition, gates, measurement, entanglement	Everything
2. Optimization on quantum	Ising / QUBO, adiabatic idea, QAOA, VQE	The solver
3. Portfolio to QUBO	Markowitz, binary encoding, penalties, costs, ESG, multi-period	Problem layer
4. Noise	Decoherence, noise models, noisy simulation	NISQ+ realism
5. Error mitigation	Dynamical decoupling, ZNE, PEC	Ablation study
6. Error correction	Stabilizers, surface code, threshold, Stim	Logical qubits
7. Resource estimation	Qubit counts, code distance, simulation strategy	Report (a)
8. Quantum advantage	Approximation ratio, sample complexity, Grover, QAE	Theory part

Chapter	What you learn	Needed for
9. Baselines + statistics	Gurobi, stochastic programming, RL, significance tests	Report (c)
10. Stack + plan	Libraries, code skeletons, repo, dashboard, report outline	Execution

1. Quantum computing basics

1.1 From bit to qubit

A classical bit is 0 or 1. A **qubit** is a two-level quantum system whose state is a unit vector in a 2-dimensional complex vector space. The two basis states are written in Dirac (ket) notation:

$$|0\rangle = [1,\ 0]^T, \qquad |1\rangle = [0,\ 1]^T$$

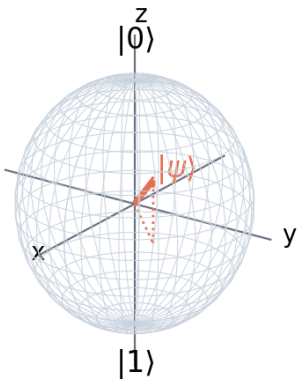
$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle, \qquad \alpha, \beta \in \mathbb{C}, \qquad |\alpha|^2 + |\beta|^2 = 1$$

Superposition means the qubit holds both amplitudes at once. **Measurement** forces a classical outcome: you see 0 with probability |alpha|^2 and 1 with probability |beta|^2 (the Born rule), and the state collapses. You can never read alpha and beta directly, only estimate them by repeating the experiment many times (**shots**).

$$P(0) = |\alpha|^2, \qquad P(1) = |\beta|^2$$

Any single-qubit state can be drawn as a point on the **Bloch sphere** using two angles:

$$|\psi\rangle = \cos\frac{\theta}{2}\,|0\rangle + e^{i\varphi}\sin\frac{\theta}{2}\,|1\rangle$$



Bloch sphere. North pole = |0>, south pole = |1>. Gates are rotations of this arrow.

1.2 Many qubits and the exponential state space

n qubits are described by 2^n complex amplitudes (tensor product space). This exponential size is both the source of quantum power and the reason classical simulation becomes impossible beyond about 30-50 qubits.

$$|\Psi\rangle = \sum_{x \in \{0,1\}^n} c_x\,|x\rangle, \qquad \sum_x |c_x|^2 = 1$$

$$\text{memory (statevector)} \;=\; 2^n \times 16 \text{ bytes} \;\Rightarrow\; n = 30:\ 16 \text{ GB}$$

1.3 Quantum gates

Gates are **unitary** matrices ($U^{dagger}U = I$), so they are reversible and preserve total probability. Memorise these:

Gate	Matrix (rows)	Effect	Used in project for
X (NOT)	[[0,1],[1,0]]	Flips |0> and |1>	Initial states, mixers
Z	[[1,0],[0,-1]]	Phase flip on |1>	Cost Hamiltonian is made of Z operators
H (Hadamard)	(1/sqrt2)[[1,1],[1,-1]]	|0> to (|0>+|1>)/sqrt2, equal superposition	Start of QAOA
$R_X(t)$	[[cos t/2, -i sin t/2],[-i sin t/2, cos t/2]]	Rotation about X axis	QAOA mixer
$R_Z(t)$	diag($e^{-it/2}$, $e^{it/2}$)	Rotation about Z axis	Linear terms h_i
CNOT	4x4: flips target if control = 1	Entangles two qubits	Builds ZZ interactions
$R_{ZZ}(t)$	exp(-i t/2 Z(x)Z)	Two-qubit phase interaction	Quadratic terms J_{ij}

Two crucial identities. The ZZ rotation (the workhorse of QAOA) is built from two CNOTs and one R_Z:

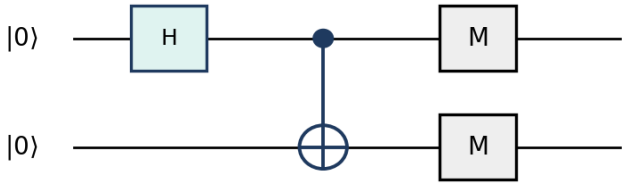
$$R_{ZZ}(\theta) = \mathrm{CNOT}_{01}\,(I \otimes R_Z(\theta))\,\mathrm{CNOT}_{01}$$

$$R_X(\theta) = e^{-i\theta X/2}, \qquad R_Z(\theta) = e^{-i\theta Z/2}, \qquad R_{ZZ}(\theta) = e^{-i\theta\, Z\otimes Z/2}$$

1.4 Entanglement and your first circuit

Entangled states cannot be written as a product of single-qubit states. The Bell state is the standard example: measuring one qubit instantly fixes the other.

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}} = \mathrm{CNOT}\,(H \otimes I)\,|00\rangle$$



Bell-state circuit. Outcomes 00 and 11 each appear with probability 1/2, never 01 or 10.

```
from qiskit import QuantumCircuit
from qiskit.primitives import StatevectorSampler
qc = QuantumCircuit(2, 2)
qc.h(0); qc.cx(0, 1); qc.measure([0, 1], [0, 1])
job = StatevectorSampler().run([qc], shots=1000)
print(job.result()[0].data.c.get_counts())   # ~{'00': 500, '11': 500}
```

1.5 Observables and expectation values (key for optimization)

An **observable** is a Hermitian matrix O (for example Z or a sum of Z terms). Its **expectation value** is the average result over infinitely many measurements. In your project, the *cost function* will be an observable, and the variational algorithm minimises its expectation.

$$\langle O \rangle = \langle \psi | O | \psi \rangle \approx \frac{1}{N_{\text{shots}}} \sum_{s=1}^{N_{\text{shots}}} o_s, \qquad \text{std. error} \sim \frac{1}{\sqrt{N_{\text{shots}}}}$$

Checkpoint (end of Day 1 morning)
You should be able to: (1) explain superposition and measurement in two sentences; (2) write the Bell circuit from memory; (3) say why n qubits need 2^n numbers to simulate; (4) explain what a shot is and why we need thousands of them. If yes, move on.

2. Quantum optimization: Ising, QUBO, QAOA, VQE

2.1 QUBO and Ising: the language quantum optimizers speak

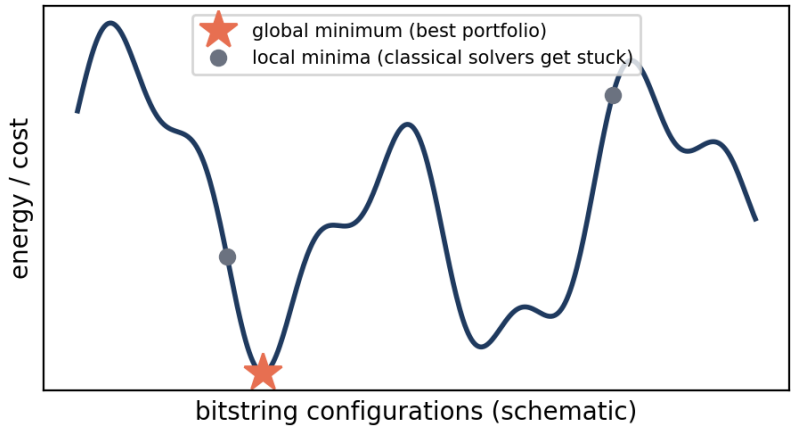
QUBO (Quadratic Unconstrained Binary Optimization) minimises a quadratic function of binary variables. **Ising** is the same problem written with spins s = +1 or -1, which is what a quantum Hamiltonian naturally encodes. They convert into each other with x = (1 - s)/2.

$$\text{QUBO}: \quad \min_{x\in\{0,1\}^n} x^T Q x = \sum_i Q_{ii} x_i + \sum_{i<j} Q_{ij} x_i x_j$$

$$\text{Ising}: \quad \min_{s\in\{\pm 1\}^n} E(s) = \sum_i h_i s_i + \sum_{i<j} J_{ij} s_i s_j$$

In quantum form each spin becomes a Pauli-Z operator, giving the **cost Hamiltonian**. Its lowest-energy eigenstate encodes the optimal solution, which is why finding the ground state solves the problem:

$$H_C = \sum_i h_i Z_i + \sum_{i<j} J_{ij} Z_i Z_j, \qquad H_C|x\rangle = E(x)|x\rangle$$



Optimization = finding the lowest point of a rugged energy landscape. Quantum methods hope to tunnel or interfere their way out of local minima.

2.2 The adiabatic idea (intuition behind QAOA)

Start in the easy ground state of a mixer Hamiltonian H_M (all qubits in |+>), then slowly morph toward H_C. If you go slowly enough the system stays in the ground state (adiabatic theorem), ending in the answer. QAOA is a **digitised, short version** of this.

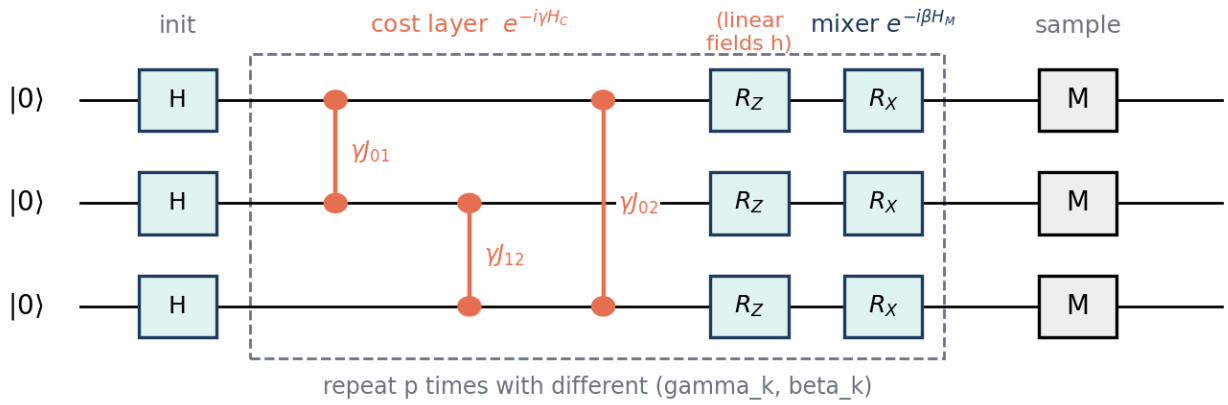
$$H(t) = \left(1 - \tfrac{t}{T}\right) H_M + \tfrac{t}{T} H_C, \qquad H_M = -\sum_i X_i$$

2.3 QAOA (Quantum Approximate Optimization Algorithm)

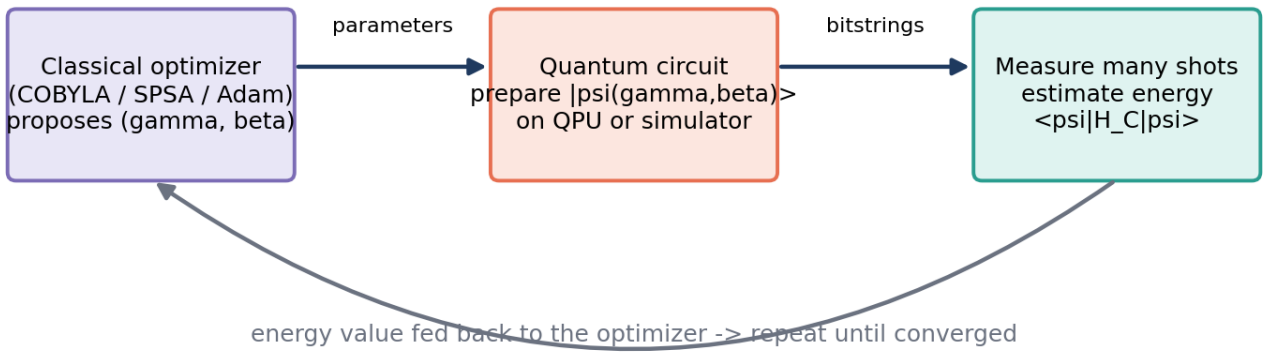
QAOA (Farhi, Goldstone, Gutmann 2014) alternates two operations p times. Parameters are the 2p angles gamma and beta. The circuit is shallow, which suits noisy hardware.

$$|\psi(\vec{\gamma},\vec{\beta})\rangle = \prod_{k=1}^{p} [e^{-i\beta_k H_M} e^{-i\gamma_k H_C}] \, |+\rangle^{\otimes n}$$

$$F_p(\vec{\gamma},\vec{\beta}) = \langle\psi(\vec{\gamma},\vec{\beta})| H_C |\psi(\vec{\gamma},\vec{\beta})\rangle, \qquad \min_{\vec{\gamma},\vec{\beta}} F_p$$



One QAOA layer on 3 qubits (a fully connected 3-asset problem). Cost layer = ZZ gates for every nonzero J_ij; mixer = X rotations.



The hybrid quantum-classical loop. This loop is the heart of your solver.

Why it is hybrid: the quantum computer only prepares states and measures; a classical optimizer (COBYLA, SPSA, Adam with parameter-shift gradients) updates the angles. Typical depth p = 1 to 5. Larger p can only match or improve the ideal result but adds noise and a harder optimisation landscape (the **barren plateau** problem: gradients vanish exponentially with qubit count for random deep circuits).

Practical QAOA tricks you should use
Warm-start from the classical relaxed (continuous Markowitz) solution; initialise angles by linear-ramp (annealing-inspired) schedule; use CVaR as the objective (average of the best alpha-fraction of shots) which converges faster; scale penalty weights carefully so constraint terms do not dominate.

2.4 VQE and adaptive ansatz

The **Variational Quantum Eigensolver** uses the same hybrid loop but with a free-form parameterised circuit U(theta) instead of the problem-structured QAOA circuit. The variational principle guarantees the energy is never below the true ground energy:

$$E(\vec{\theta}) = \langle 0|U^{\dagger}(\vec{\theta})\, H_C\, U(\vec{\theta})|0\rangle \;\geq\; E_0$$

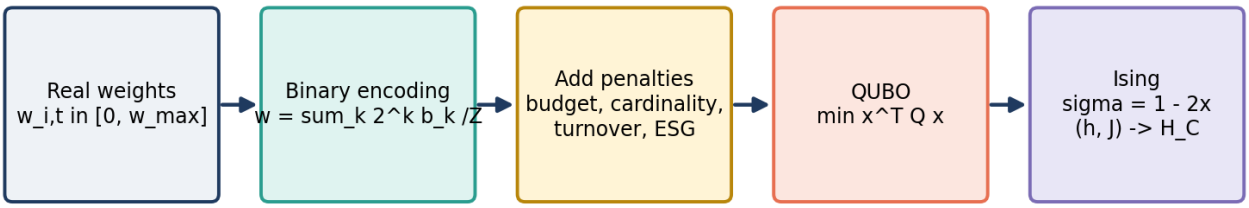
Adaptive ansatz (ADAPT-VQE / ADAPT-QAOA): start from a trivial circuit and greedily add the operator from a pool whose energy gradient is largest, re-optimising each time. This gives shorter circuits than a fixed ansatz. For your project: build the pool from single-qubit X/Y rotations and ZZ terms; the gradient of operator A at the current state is:

$$\frac{\partial E}{\partial \theta_A}\Big|_{\theta_A = 0} = i\,\langle \psi|\,[H_C, A]\,|\psi\rangle$$

2.5 Quantum annealing (for the Ocean SDK)

D-Wave annealers physically implement the adiabatic idea on thousands of qubits. Your stack mentions **Ocean SDK**; use it mainly for (a) building the QUBO with `dimod`, (b) running classical simulated annealing as a baseline (`neal`), and (c) optionally the hybrid solver if you have a free D-Wave Leap account.

3. Portfolio optimization as QUBO



Each binary variable x_j becomes one qubit. Q_jk becomes a ZZ interaction.

The formulation pipeline. Each stage is derived below.

3.1 The classical starting point (Markowitz)

Choose weights w across N assets to balance expected return against risk. Here mu is the expected return vector, Sigma the covariance matrix, q the risk-aversion parameter.

$$\min_{w} \; q\, w^T \Sigma w - \mu^T w \qquad \text{s.t.} \;\; \mathbf{1}^T w = 1, \;\; w \geq 0$$

Return metrics you will need: portfolio return $r_p = w^T r$, volatility $\text{sigma}_p = \text{sqrt}(w^T \text{ Sigma } w)$, and the Sharpe ratio:

$$\text{Sharpe} = \frac{\mathbb{E}[r_p - r_f]}{\sigma_p} \cdot \sqrt{252} \;\; \text{(annualised from daily)}$$

3.2 Binary encoding of weights

Qubits are binary, but weights are continuous. Use K bits per asset (fixed-point encoding). With K = 3 each weight takes 8 levels. Total qubits = N x K per period.

$$w_i = \frac{1}{Z} \sum_{k=0}^{K-1} 2^k \, b_{i,k}, \qquad b_{i,k} \in \{0, 1\}, \qquad Z = \text{normaliser}$$

$$n_{\text{qubits}} = N \times K \times T \quad \text{(assets} \times \text{bits} \times \text{periods)}$$

An alternative that needs far fewer qubits is **selection encoding**: one bit per asset (hold it or not, equal-weight among selected) with a cardinality constraint. This is the form most QAOA finance papers use and is ideal for your simulatable tier.

3.3 Turning constraints into penalties

QUBO has no constraints, so each constraint becomes a squared penalty that is zero when satisfied and positive otherwise. Choose the penalty weight lambda larger than the largest possible objective gain from violating the constraint.

$$\text{Budget}: \quad \lambda_B \left(\sum_{i,k} \frac{2^k}{Z} b_{i,k} - 1\right)^2$$

$$\text{Cardinality (hold exactly } m \text{ assets)}: \quad \lambda_C \left(\sum_i x_i - m\right)^2$$

$$\text{Inequality } \sum_i a_i x_i \le c: \quad \lambda \left(\sum_i a_i x_i + \sum_j 2^j s_j - c\right)^2 \quad (\text{slack bits } s_j)$$

Constraint type	How to handle	Comment
Hard (must hold)	Large lambda, or restrict the quantum mixer to preserve it (XY mixer keeps Hamming weight fixed)	XY mixer removes the cardinality penalty entirely
Soft (preferred)	Moderate lambda, treated as a trade-off	Good for risk target, tracking error
Time-varying	lambda_t and bounds depend on period t (e.g. tighter volatility cap in a high-VIX regime)	This is your 'dynamic constraints'

3.4 Transaction costs and ESG

Transaction cost penalises changing positions between periods t-1 and t. Linear costs are an absolute value (not quadratic), so use a quadratic proxy or the binary-difference trick; the quadratic form is directly QUBO-compatible:

$$\text{Cost}_t = c \sum_i |w_{i,t} - w_{i,t-1}| \quad \approx \quad \kappa \sum_i (w_{i,t} - w_{i,t-1})^2$$

ESG filters come in two flavours. A *hard filter* removes assets below an ESG threshold before building the QUBO (reduces qubits, free). A *soft constraint* requires the portfolio-average ESG score to exceed a floor, which is linear and therefore becomes a squared penalty:

$$\text{ESG floor}: \quad \lambda_E \left(\sum_i e_i w_i - E_{\min} + \text{slack}\right)^2$$

3.5 Multi-period, time-dependent QUBO

Stack T periods. Variables are $w_{i,t}$; the objective sums per-period risk-return terms, transaction costs linking neighbouring periods, and time-varying penalties. The result is one big QUBO whose interaction graph has *intra-period* blocks (risk covariance) and *inter-period* links (transaction cost).

$$\min_{\{w_t\}} \sum_{t=1}^{T} \left[q_t \, w_t^T \Sigma_t w_t - \mu_t^T w_t + \kappa \| w_t - w_{t-1} \|^2 \right] + \sum_t \text{Penalties}_t(\lambda_t)$$

Rolling re-optimisation (used in the dashboard): at each new market tick, update mu and Sigma, rebuild the QUBO for the next horizon, warm-start the QAOA angles from the previous solution, execute only the first period's trade. This is model-predictive control.

Time-dependent QUBO in the quantum-annealing sense can also mean the Hamiltonian coefficients vary during evolution: H(t) = sum of (coefficients that depend on t). For QAOA you simply produce a new (h, J) per re-optimisation step.

3.6 QUBO to Ising conversion (worked)

Substitute x = (1 - s)/2 into the QUBO. The result gives h and J up to an irrelevant constant:

$$J_{ij} = \frac{Q_{ij}}{4}, \qquad h_i = -\frac{Q_{ii}}{2} - \sum_{j \neq i} \frac{Q_{ij}}{4}, \qquad \text{offset} = \sum_i \frac{Q_{ii}}{2} + \sum_{i<j} \frac{Q_{ij}}{4}$$

```
import numpy as np, dimod
from qiskit_optimization import QuadraticProgram
from qiskit_optimization.converters import QuadraticProgramToQubo

def selection_qubo(mu, Sigma, q, m, lam):
    n = len(mu); Q = q*Sigma.copy() - np.diag(mu)          # risk - return
    Q += lam*(np.ones((n, n)) - 2*m*np.eye(n))             # lam*(sum x - m)^2, using x^2 = x
    return Q                                               # constant lam*m^2 dropped
bqm = dimod.BinaryQuadraticModel.from_numpy_matrix(selection_qubo(mu, Sigma, 3.0, 4, 5.0))
h, J, offset = bqm.to_ising()                              # Ising form for the circuit
```

Sanity test you must always run
For N <= 20 binary variables, brute-force all 2^n bitstrings and confirm your QUBO minimum equals the Gurobi / classical optimum. A wrong sign or penalty scale is the number one bug in QUBO projects.

4. Noise: what makes real quantum computers hard

4.1 NISQ and sources of error

NISQ = Noisy Intermediate-Scale Quantum (Preskill 2018): 50 to a few thousand qubits, no error correction, circuits limited to roughly 10^2-10^3 two-qubit gates. The problem statement says **NISQ+**, meaning NISQ-era noise models combined with partial or early error correction.

Error source	Meaning	Typical value today
T1 (relaxation)	|1> decays to |0> (energy loss)	50-300 microseconds
T2 (dephasing)	Loss of phase between |0> and |1>	50-200 microseconds
Gate error	Imperfect pulses	1-qubit ~0.02-0.1%, 2-qubit ~0.1-1%
Readout error	Measuring 0 reports 1 or vice versa	~0.5-3%
Crosstalk	Operations disturb neighbours	hardware dependent

4.2 Mathematical noise models

A noisy state is a **density matrix** rho and noise is a **channel** described by Kraus operators K_k:

$$\rho \longrightarrow \mathcal{E}(\rho) = \sum_k K_k \rho K_k^\dagger, \qquad \sum_k K_k^\dagger K_k = I$$

$$\text{Depolarizing}: \quad \mathcal{E}(\rho) = (1-p)\,\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z)$$

$$\text{Idle decay}: \quad \rho_{11}(t) = \rho_{11}(0)\, e^{-t/T_1}, \qquad \text{coherence} \propto e^{-t/T_2}$$

Why depth matters: if each gate succeeds with probability (1 - epsilon), a circuit with G gates has total fidelity roughly:

$$F \approx (1-\varepsilon)^G \approx e^{-\varepsilon G}$$

With epsilon = 0.5% and G = 500 two-qubit gates, F is about 8%, so the signal is nearly drowned in noise. A dense 30-qubit QUBO has 435 ZZ terms per layer, which immediately shows why dense portfolio problems are hard on NISQ devices.

4.3 How to simulate noise in Qiskit

```
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error, thermal_relaxation_error, ReadoutError

nm = NoiseModel()
nm.add_all_qubit_quantum_error(depolarizing_error(0.001, 1), ['rx', 'rz', 'h', 'sx'])
nm.add_all_qubit_quantum_error(depolarizing_error(0.01, 2),  ['cx', 'rzz'])
nm.add_all_qubit_readout_error(ReadoutError([[0.98, 0.02], [0.03, 0.97]]))
noisy = AerSimulator(noise_model=nm)                # or AerSimulator.from_backend(fake_backend)
```

Better realism: `AerSimulator.from_backend(FakeSherbrooke())` (from `qiskit_ibm_runtime.fake_provider`) copies a real IBM device's measured noise and connectivity. Connectivity forces extra SWAP gates (transpilation), another reason dense problems suffer.

5. Error mitigation: DD, ZNE and PEC

Mitigation reduces the effect of noise in the *measured result* without encoding qubits (unlike error correction). It costs extra shots, not extra qubits. Your ablation (report item b) tests every combination.

Technique	Idea	Cost	Fixes	Weakness
Dynamical decoupling (DD)	Insert pulse sequences (X-X, XY4) into idle windows to cancel dephasing and crosstalk	Tiny	Idle errors	Cannot fix gate errors
Zero-noise extrapolation (ZNE)	Run at amplified noise levels, extrapolate to zero noise	3-5x shots	Moderate noise	Bias, variance blow-up
Probabilistic error cancellation (PEC)	Invert noise as a quasi-probability mix of noisy operations	Exponential in gate count	Unbiased in theory	Needs accurate noise model
Readout mitigation (M3, TREX)	Invert measured confusion matrix	Small	Measurement errors	Only readout

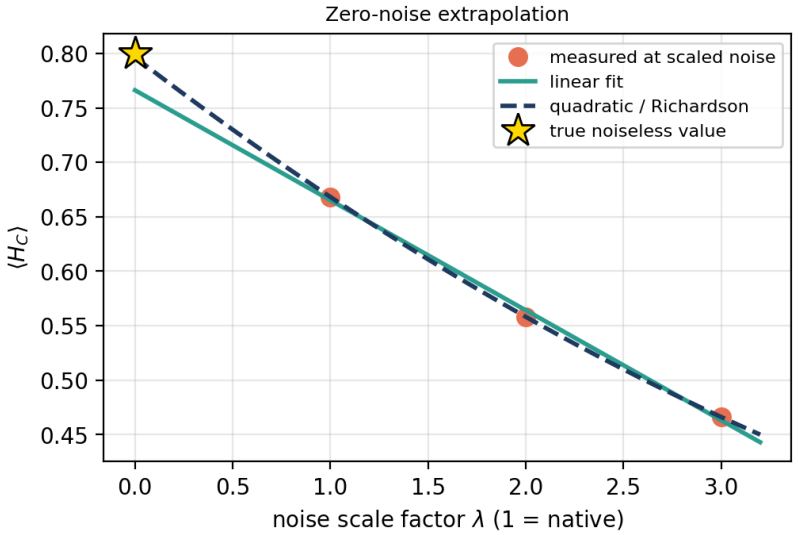
5.1 Dynamical decoupling

A qubit idling accumulates unwanted phase. Flipping it with X pulses in a symmetric pattern makes the accumulated error cancel (echo effect), like spin echo in NMR. Qiskit applies it as a transpiler pass (`PadDynamicalDecoupling`).

$$\text{idle } \tau \longrightarrow \text{ wait } \tfrac{\tau}{2} \to X \to \text{wait } \tfrac{\tau}{2} \to X \quad (X^2 = I)$$

5.2 Zero-noise extrapolation

Deliberately **fold** gates (replace G by G G^{dagger} G) to scale noise by lambda = 1, 3, 5..., measure the expectation at each, fit a curve, and read off the value at lambda = 0.



ZNE: extrapolate measured values back to the noiseless point. Richardson (polynomial) fit is common; exponential fit often better for depolarizing noise.

$$E(\lambda) \approx E_0\, e^{-a\lambda} \;\Rightarrow\; E_0 = \lim_{\lambda \to 0} E(\lambda)$$

$$\text{Richardson for } \lambda = 1, 3: \quad E_0 \approx \frac{3\,E(1) - E(3)}{2}$$

5.3 Probabilistic error cancellation

If the noisy implementation of an ideal gate G is a channel N, learn N and write the *inverse* as a signed combination of implementable noisy operations O_j. Sample operations according to $|q_j|$, multiply results by $\text{sign}(q_j)$ times gamma. The result is unbiased but variance grows with the **sampling overhead** gamma:

$$G = \sum_j q_j \mathcal{O}_j, \qquad \gamma = \sum_j |q_j| \geq 1, \qquad \gamma_{\text{total}} = \prod_{g=1}^{G} \gamma_g \approx e^{2\varepsilon G}$$

$$\text{shots needed} \propto \frac{\gamma_{\text{total}}^2}{\delta^2} \text{ to reach accuracy } \delta$$

This exponential cost is exactly why PEC works on shallow circuits only and why error *correction* is ultimately required. Mention this in your report: it connects NISQ mitigation (Section 5) to fault tolerance (Section 6).

5.4 Ablation design (report item b)

Run	DD	Readout mit.	ZNE	PEC	Metric to record
A (baseline)	-	-	-	-	Approx. ratio, best-bitstring energy, Sharpe
B	yes	-	-	-	same
C	yes	yes	-	-	same
D	yes	yes	yes	-	same + shots used
E	yes	yes	-	yes	same + shots used
F (full)	yes	yes	yes	yes	same + shots used
Reference	ideal noiseless simulator				upper bound

Use **mitiq** (`pip install mitiq`) for ZNE and PEC: `zne.execute_with_zne(circuit, executor, ...)` and `pec.execute_with_pec(...)`. Report the *shots-matched* comparison, otherwise mitigation looks better simply because it uses more shots.

6. Quantum error correction and logical qubits

6.1 Why correction is harder than for classical bits

Classical error correction copies a bit (repetition code). Quantum states cannot be cloned (no-cloning theorem), and measuring destroys them, and there are two error types (bit flip X and phase flip Z). The solution: spread information over many **physical** qubits and measure only **parity checks** (stabilizers) that reveal errors without revealing the data.

6.2 Repetition code (the warm-up)

Encode one logical qubit in 3 physical ones, protecting against bit flips. Stabilizers Z_1Z_2 and Z_2Z_3 detect a flip and tell you which qubit to fix, without measuring the encoded value:

$$|\bar{0}\rangle = |000\rangle, \quad |\bar{1}\rangle = |111\rangle, \qquad \text{syndrome} = (Z_1Z_2,\ Z_2Z_3)$$

Error	Z1Z2	Z2Z3	Correction
none	+1	+1	-
X on q1	-1	+1	flip q1
X on q2	-1	-1	flip q2
X on q3	+1	-1	flip q3

6.3 Stabilizer codes

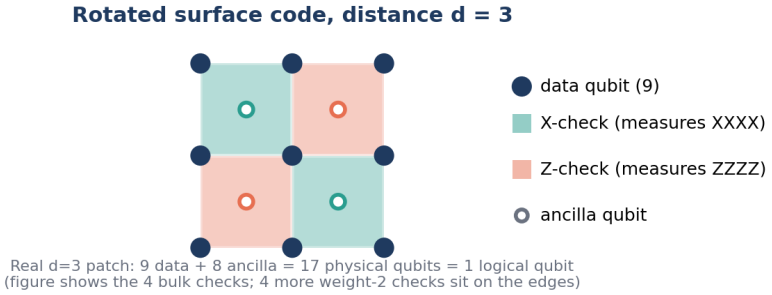
A stabilizer code is defined by a group of commuting Pauli operators S. The code space is the +1 eigenspace of all of them. An error that anticommutes with a stabilizer flips its measured value (a **syndrome**). A **decoder** (classical software) reads syndromes and guesses the most likely error.

$$\mathcal{C} = \{|\psi\rangle :\ S_i|\psi\rangle = +|\psi\rangle \ \ \forall i\}, \qquad [S_i, S_j] = 0$$

$$[[n, k, d]]:\ n \text{ physical qubits},\ k \text{ logical qubits},\ \text{distance } d,\ \ \text{corrects } \lfloor (d-1)/2 \rfloor \text{ errors}$$

6.4 The surface code

The surface code is the leading candidate for fault tolerance: it needs only nearest-neighbour interactions on a 2D grid, and has a high error threshold of roughly 1%. Data qubits sit on a grid; X-type and Z-type checks (4-body) tile the grid in a checkerboard.



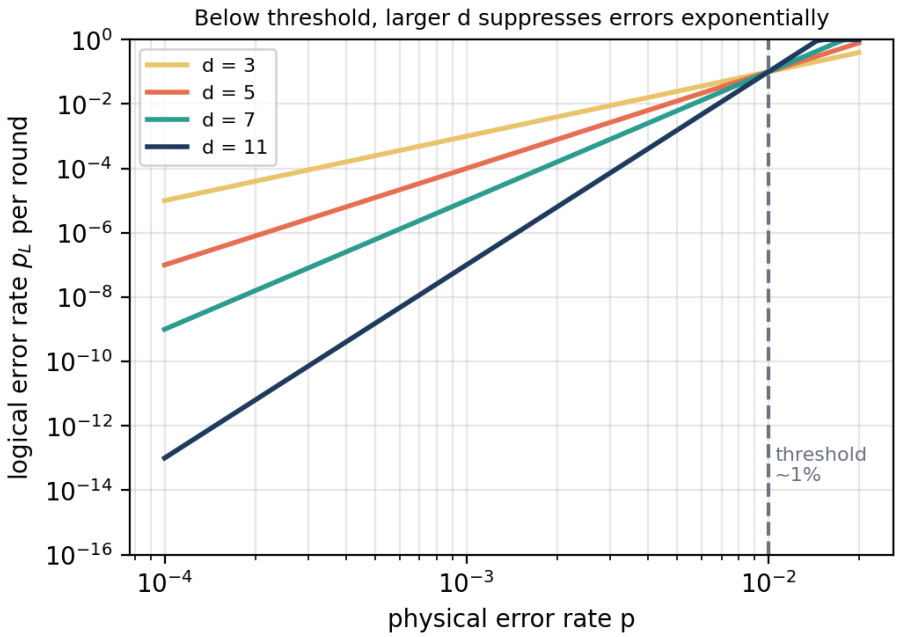
Distance-3 rotated surface code patch. Real patches add edge checks. Bigger d means a bigger grid.

$$\text{Rotated surface code}: \quad [[\, d^2,\ 1,\ d\,]], \qquad n_{\text{phys}} = d^2\ (\text{data}) + (d^2-1)\ (\text{ancilla}) = 2d^2 - 1$$

One **QEC round** measures all checks once (about 1 microsecond on superconducting hardware). A **logical error** happens when physical errors form a chain connecting two opposite boundaries of the patch, which requires at

least (d+1)/2 errors. Therefore below threshold:

$$p_L \approx A\left(\frac{p}{p_{\text{th}}}\right)^{\lceil (d+1)/2 \rceil}, \qquad A \sim 0.03 - 0.1, \quad p_{\text{th}} \approx 10^{-2}$$



Logical error rate vs physical error rate (formula plot). Each step d to d+2 multiplies suppression by p/p_th.

6.5 Logical operations

Computing on logical qubits is possible, but it is the expensive part:

- **Clifford gates** (H, S, CNOT) are done by **transversal** operations or **lattice surgery** (merging and splitting patches). Cost: about d QEC rounds per logical CNOT.
- **Non-Clifford gates** (T gate, needed for arbitrary rotations such as R_Z(gamma) in QAOA) require **magic state distillation** or cultivation: factories that consume many physical qubits and time. Each arbitrary rotation is synthesised from T gates.
- T-count for an arbitrary rotation to precision epsilon is about 1.15 $\log_2$(1/epsilon) + 9 (Ross-Selinger synthesis). This is why QAOA with dense ZZ layers is costly on fault-tolerant machines.

$$N_T \approx 1.15 \log_2\left(\frac{1}{\varepsilon_{\text{rot}}}\right) + 9 \;\text{ per arbitrary rotation}$$

6.6 Stim and PyMatching in practice

Stim (Gidney 2021) is a very fast stabilizer-circuit simulator that has built-in surface code generators. **PyMatching** is the decoder (minimum-weight perfect matching). Together they give you measured p_L(d, p), the data behind your logical-qubit analysis.

```
import stim, pymatching, numpy as np

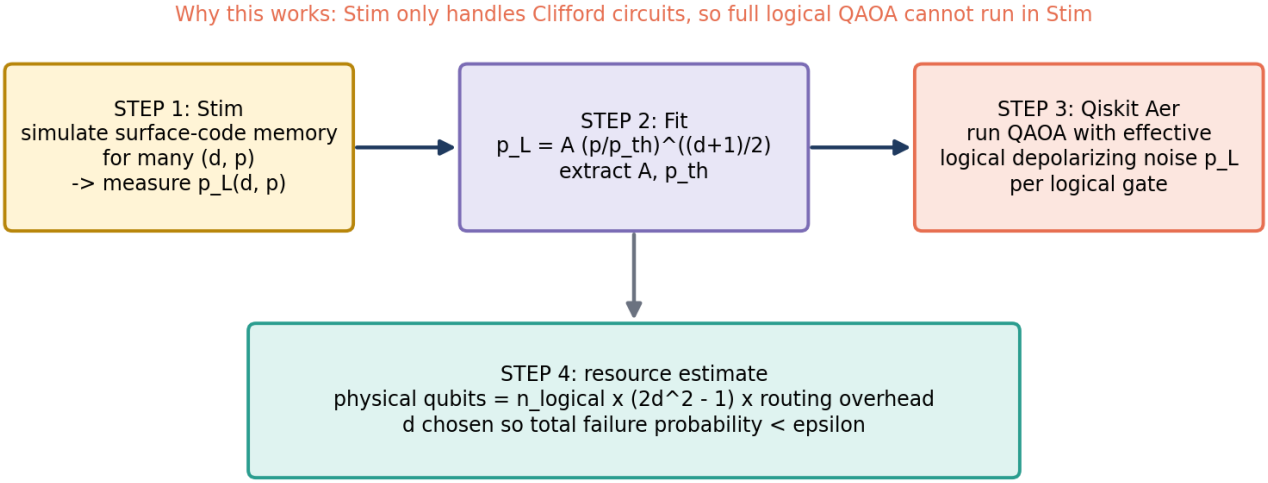
def logical_error_rate(d, p, shots=20000):
    c = stim.Circuit.generated("surface_code:rotated_memory_z", distance=d, rounds=d,
                               after_clifford_depolarization=p, after_reset_flip_probability=p,
                               before_measure_flip_probability=p, before_round_data_depolarization=p)
    dem = c.detector_error_model(decompose_errors=True)
    matcher = pymatching.Matching.from_detector_error_model(dem)
    det, obs = c.compile_detector_sampler().sample(shots, separate_observables=True)
    pred = matcher.decode_batch(det)
    return np.mean(np.any(pred != obs, axis=1))

for d in (3, 5, 7):
    print(d, [logical_error_rate(d, p) for p in (0.001, 0.003, 0.006)])
```

Plot the results on a log-log axis (like the formula plot above). The crossing point of the curves for different d is your **measured threshold**. Fit A and p_{th} with least squares on log p_L.

7. Resource estimation and the simulation strategy

7.1 The method



The recommended way to run 'logical qubits' in a 5-day project: measure the logical error rate with Stim, then apply it as an effective noise model in the QAOA simulation.

Step 1. Count logical resources of your QAOA circuit: n logical qubits, depth, number of two-qubit gates, number of rotations. **Step 2.** Choose the code distance d so that the total failure probability is small. **Step 3.** Convert to physical qubits and runtime.

$$\text{Volume } V = n_{\text{logical}} \times D_{\text{logical}} \qquad \text{(logical qubits} \times \text{logical time steps)}$$

$$p_L(d) \cdot V \le \varepsilon_{\text{tot}} \;\Rightarrow\; d = \min\{d \text{ odd}: A(p/p_{\text{th}})^{(d+1)/2} \le \frac{\varepsilon_{\text{tot}}}{V}\}$$

$$N_{\text{phys}} = n_{\text{logical}}\,(2d^2 - 1) \cdot f_{\text{route}} + N_{\text{factories}} \cdot (\text{qubits per factory}), \qquad t_{\text{run}} = D_{\text{logical}} \cdot d \cdot t_{\text{round}} \cdot N_{\text{shots}}$$

7.2 Worked example (computed for you)

Assume N = 30 assets, K = 3 bits per weight, T = 4 periods gives n = 360 logical qubits (the second row set). A smaller 12 x 10 problem gives n = 120. Assume A = 0.1, p_{th} = 1%, total failure budget epsilon = 1%, dense interaction graph so about n sequential two-qubit layers per QAOA layer, and routing overhead factor 2. Magic-state factories are *not* included, so these are lower bounds.

Physical error p	Logical gate steps D	Volume V	Distance d	Physical qubits (x2 routing)	Runtime (10^4 shots, 1 us rounds)
p=0.001, n=120, depth p=3	360	43,200	11	57,840	40 s
p=0.0001, n=120, depth p=3	360	43,200	5	11,760	18 s
p=0.001, n=360, depth p=5	1,800	648,000	13	242,640	234 s
p=0.0001, n=360, depth p=5	1,800	648,000	7	69,840	126 s

Reading the table: even at a good physical error rate of 10^{-3}, a realistic multi-period portfolio QUBO needs on the order of 10^5 physical qubits, far beyond today's few-thousand-qubit devices. This is a legitimate and valuable finding: it quantifies the gap between the problem and the hardware. Add magic-state factory estimates and

compare with published tools (Azure Quantum Resource Estimator, Qualtran, Gidney-Ekera methodology).

7.3 Alternative qubit-saving formulations

Idea	Effect
Selection encoding (1 bit per asset)	n = N x T instead of N x K x T
Pre-filter by ESG and liquidity	Reduces N before encoding
Sparse covariance (factor model, k factors)	Cuts ZZ terms from $N^2/2$ to about N k
Decompose across periods (rolling horizon)	Smaller sub-problems, QAOA only on the hardest block
Hamming-weight-preserving XY mixer	Removes cardinality penalty, fewer terms

7.4 What you actually run (three simulation tiers)

Tier	Tool	Size	What it shows
A: ideal	Aer statevector	up to ~24 qubits	Algorithm quality without noise (upper bound)
B: NISQ noisy	Aer density matrix / trajectories + noise model	10-14 qubits (exact), more with MPS	Effect of noise, error-mitigation ablation
C: logical	Stim + PyMatching, then Aer with logical noise p_L	Stim: 1000s of qubits; QAOA: 10-14 logical	Fault-tolerant regime and the resource table

Why Tier C is still honest
In fault-tolerant operation, noise on a logical qubit is well described by a small depolarizing channel with probability p_L per logical operation (this is the standard assumption of resource-estimation papers). You are not faking error correction: you are measuring p_L from a true circuit-level surface-code simulation and propagating it. State this assumption explicitly.

8. Quantum advantage: what can be proven

This chapter gives you the theory section of the report. Each result below states clearly *what is proved* and *what is only empirical*.

8.1 Defining advantage

- **Solution quality:** for equal wall-clock or equal queries, quantum returns a better objective value.
- **Sample / query complexity:** quantum needs asymptotically fewer samples or oracle calls for the same accuracy.
- **Time-to-solution (TTS):** time to reach the optimum with 99% confidence. This is the fair metric for heuristics.

$$\mathrm{TTS}_{99} = t_{\mathrm{run}} \cdot \frac{\ln(1-0.99)}{\ln(1-p_s)}, \qquad p_s = \text{success probability of one run}$$

8.2 Approximation ratio of QAOA

The approximation ratio r measures how close the found cost is to the optimum. For minimisation, with E_{min} and E_{max} the best and worst energies:

$$r = \frac{E_{\max} - \langle H_C \rangle}{E_{\max} - E_{\min}} \in [0,1] \qquad (r = 1 \text{ optimal})$$

Proved results you can cite: for MaxCut on 3-regular graphs, QAOA at p=1 guarantees r >= 0.6924 (Farhi et al. 2014); the best classical polynomial algorithm (Goemans-Williamson) guarantees 0.878 for general graphs. QAOA with p growing large converges to r = 1 (adiabatic limit) but with no proven polynomial depth. For **dense, weighted QUBO from portfolio problems** (Sherrington-Kirkpatrick-like), there are results showing QAOA at constant p matches a limiting value, from Farhi et al. 2019, but **no guarantee beats classical algorithms**. State this explicitly.

$$\text{Noise bound (depolarizing, G gates):} \quad \langle H_C \rangle_{\text{noisy}} \approx F \langle H_C \rangle_{\text{ideal}} + (1-F)\frac{\mathrm{Tr} H_C}{2^n}, \qquad F \approx e^{-\varepsilon G}$$

This formula is the key analytic tool for your noise analysis: the noisy energy interpolates between the ideal energy and the maximally mixed state value (random guessing). It predicts the depth at which QAOA quality falls to random, and is easy to verify numerically.

8.3 Grover search and QAOA-type speedups

For unstructured search over 2^n candidates Grover's algorithm needs O(sqrt(2^n)) oracle calls, a proven quadratic speedup over classical O(2^n), and it is optimal. Applied to QUBO minimisation (Durr-Hoyer minimum finding):

$$T_{\text{classical}} = O(2^n), \qquad T_{\text{Grover}} = O(2^{n/2})$$

Caveat: modern classical solvers (branch-and-bound, Gurobi) do far better than brute force on structured problems, and error-corrected Grover has huge constant overheads. A quadratic speedup over a 2^n algorithm often disappears once the quantum overhead is included (Babbush et al. 2021, 'Focus beyond quadratic speedups').

8.4 Quantum amplitude estimation (the provable finance speedup)

Estimating expected values (VaR, CVaR, option prices, portfolio expected loss) by Monte Carlo needs M samples to reach error epsilon. Quantum amplitude estimation (QAE) achieves the same accuracy with quadratically fewer queries:

$$M_{\mathrm{MC}} = O\left(\frac{1}{\varepsilon^2}\right) \qquad \text{vs} \qquad M_{\mathrm{QAE}} = O\left(\frac{1}{\varepsilon}\right)$$

This is a **proven** query-complexity advantage (Brassard et al. 2002; Woerner and Egger 2019 for risk analysis). You can integrate it into your project as a *risk-evaluation module*: estimate portfolio CVaR with QAE (small simulation) and compare sample counts against classical Monte Carlo. For your report this is your strongest rigorous sample-complexity result. Caveat: loading the return distribution into the quantum state is itself costly, and QAE needs fault tolerance.

Suggested structure for your theory chapter
(1) Definitions (TTS, approximation ratio, sample complexity). (2) Proposition: noise-limited energy formula, with proof sketch. (3) Proposition: QAE vs Monte Carlo sample complexity, with numerical verification. (4) Cited results for QAOA p=1 and Grover. (5) Explicit statement of what is NOT proven. (6) Empirical results for your instances.

8.5 Dequantization warning

Some quantum algorithms for portfolio optimisation (Rebentrost and Lloyd 2018; Kerenidis et al. 2019, quantum interior-point methods) promise polynomial speedups, but depend on quantum-accessible data (QRAM) and Tang's 'dequantization' results showed classical sampling algorithms matching several of them. Cite these as a caution when making any advantage claim.

9. Classical baselines and statistical testing

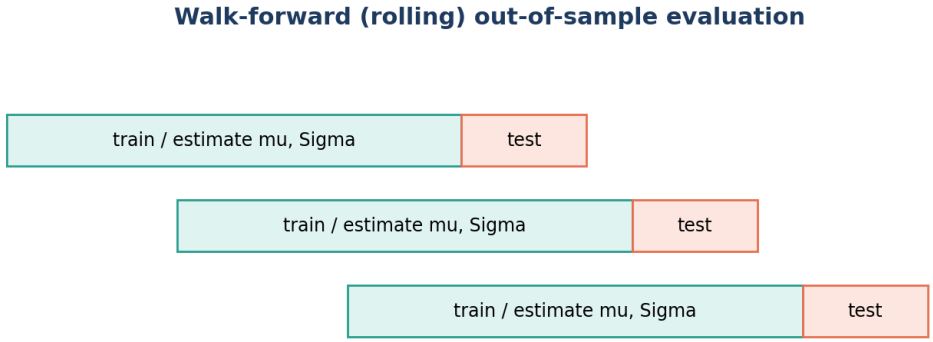
9.1 Baselines you must beat (or lose to honestly)

Baseline	What it is	How to implement
Gurobi MIQP	Exact mixed-integer quadratic solver. Needs a licence (free academic licence exists; free restricted pip licence handles about 2000 variables)	gurobipy: binary vars, quadratic objective, linear constraints
Stochastic programming	Scenarios of returns, minimise CVaR over scenarios, multi-stage	Sample S scenarios (e.g. 200), CVaR as LP (Rockafellar-Uryasev), gurobipy
Simulated annealing	Direct competitor for QUBO heuristics	dimod + neal SimulatedAnnealingSampler
Deep RL agent	Policy network chooses weights each period	PyTorch + stable-baselines3 (PPO) in a Gym portfolio environment
Simple benchmarks	Equal-weight 1/N, minimum-variance, Markowitz continuous	NumPy / cvxpy

$$\mathrm{CVaR}_\alpha = \min_{z,\, u_s \geq 0} z + \frac{1}{(1-\alpha)S} \sum_{s=1}^{S} u_s, \qquad u_s \geq -w^T r_s - z$$

Deep RL set-up: state = last 60 days of returns + macro features (VIX, yield spread) + current weights; action = target weights (softmax); reward = daily return minus transaction cost minus risk penalty (e.g. differential Sharpe). Train on pre-test periods only; use 3-5 random seeds because RL results vary heavily.

9.2 Fair evaluation protocol



Walk-forward evaluation protocol. Avoids look-ahead bias.

- **Same data, same costs, same constraints** for every method.
- **Crisis windows:** 2008-2009 (needs data from 2005 on; restrict the universe to stocks with full history) and Feb-Apr 2020. Evaluate them separately.
- **Equal compute budget:** report wall-clock and number of solver calls.
- **Metrics:** annualised return, volatility, Sharpe, max drawdown, turnover, total transaction cost, average ESG score, constraint-violation rate, and solution-quality gap to Gurobi optimum on the optimisation instances.

$$\mathrm{MDD} = \max_t \left(\frac{\max_{s \leq t} V_s - V_t}{\max_{s \leq t} V_s} \right), \qquad \mathrm{Gap} = \frac{E_{\text{method}} - E_{\text{Gurobi}}}{|E_{\text{Gurobi}}|}$$

9.3 Statistical significance tests (report item c)

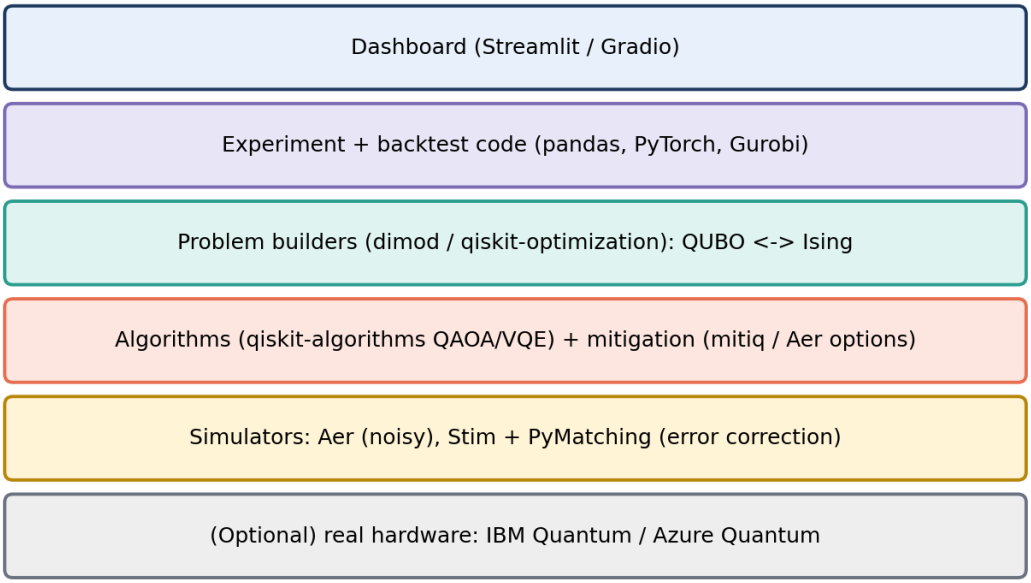
Question	Test	Tool
Is quantum Sharpe different from classical Sharpe?	Ledoit-Wolf (2008) / Jobson-Korkie-Memmel test, or paired block bootstrap of the Sharpe difference	Own code: stationary bootstrap (arch package)
Is forecast / loss series different?	Diebold-Mariano test	statsmodels / own implementation
Per-period paired returns	Paired t-test and Wilcoxon signed-rank	scipy.stats
Many strategies tried (cherry-picking)	White's Reality Check, Hansen SPA, Deflated Sharpe Ratio, Bonferroni / Holm correction	arch.bootstrap.SPA
Solver quality on random instances	Wilcoxon on gap values over 30+ random instances, effect size (Cliff's delta)	scipy
How big is the difference?	95% confidence intervals via bootstrap	numpy

$$\mathrm{DM} = \frac{\bar{d}}{\sqrt{\widehat{\mathrm{Var}}(\bar{d})}} \sim N(0,1), \qquad d_t = L(e_t^A) - L(e_t^B)$$

$$\mathrm{DSR} = \Phi\left(\frac{(\widehat{SR} - SR_0)\sqrt{T-1}}{\sqrt{1 - \hat{\gamma}_3 \widehat{SR} + \frac{\hat{\gamma}_4 - 1}{4}\widehat{SR}^2}}\right) \quad \text{(Deflated Sharpe Ratio)}$$

Daily returns are autocorrelated and fat-tailed, so use block bootstrap (block length about 10-20 days), never an iid t-test alone. Always report confidence intervals, not just p-values. Because quantum runs are stochastic, repeat each quantum configuration with at least 20 random seeds and test over seeds as well.

10. Tech stack, repository layout and the 5-day plan



Software stack from hardware up to dashboard.

10.1 Installation

```
python -m venv qenv && source qenv/bin/activate
pip install qiskit qiskit-aer qiskit-algorithms qiskit-optimization qiskit-ibm-runtime
pip install stim pymatching mitiq dimod dwave-neal dwave-ocean-sdk
pip install gurobipy cvxpy torch gymnasium stable-baselines3
pip install yfinance fredapi pandas numpy scipy statsmodels arch matplotlib streamlit
```

API names change between releases (Qiskit in particular reorganised its primitives and algorithms packages). Pin versions in `requirements.txt` on Day 1 and read the release notes if a tutorial fails to run.

10.2 Data sources

Data	Source	How
Prices	Yahoo Finance	yfinance.download(tickers, start='2005-01-01')
Macro (VIX, yield curve, rates, credit spread)	FRED	fredapi (free key): VIXCLS, T10Y2Y, DGS10, BAMLH0A0HYM2
ESG scores	MSCI / Sustainalytics open subsets (limited, check licences); fallback: Yahoo Finance sustainability field or an open dataset from Kaggle	Map to tickers, z-score, handle missing by sector median

Data hygiene: adjusted close prices, survivorship bias (use stocks that existed in 2008), forward-fill rules, covariance shrinkage (Ledoit-Wolf, sklearn `LedoitWolf`) because raw covariance from short windows is unstable.

10.3 Suggested repository structure

```
qportfolio/
  data/            download.py  features.py  esg.py
  model/           markowitz.py  qubo.py  constraints.py  multi_period.py
  quantum/         qaoa.py  vqe_adapt.py  noise.py  mitigation.py  qae_risk.py
  qec/             stim_surface.py  fit_logical.py  resources.py
  baselines/       gurobi_mip.py  stoch_prog.py  sa.py  rl_agent.py
  eval/            backtest.py  metrics.py  stats_tests.py  ablation.py
  app/             dashboard.py           (Streamlit)
  notebooks/       01_basics.ipynb ...
  report/          figures/  main.tex
  requirements.txt  README.md  Makefile    (make reproduce)
```

10.4 QAOA skeleton in Qiskit

```
from qiskit.circuit.library import QAOAAnsatz
from qiskit.quantum_info import SparsePauliOp
from qiskit_aer.primitives import EstimatorV2 as AerEstimator
from scipy.optimize import minimize
import numpy as np

def ising_to_pauli(h, J, n):
    terms = [("Z", [i], h[i]) for i in h] + [("ZZ", [i, j], w) for (i, j), w in J.items()]
    return SparsePauliOp.from_sparse_list(terms, num_qubits=n)

H = ising_to_pauli(h, J, n)
ansatz = QAOAAnsatz(H, reps=3)                          # p = 3
est = AerEstimator(options={"backend_options": {"noise_model": nm}})   # nm from Section 4.3
def cost(theta):
    return est.run([(ansatz, H, theta)]).result()[0].data.evs
theta0 = np.linspace(0.1, 1.0, ansatz.num_parameters)    # linear-ramp style init
res = minimize(cost, theta0, method="COBYLA", options={"maxiter": 200})
```

Then sample the optimised circuit, take the **best** bitstring (not just the most frequent), repair constraint violations classically if needed, and decode weights. Always report the energy of the best sampled feasible portfolio.

10.5 Dashboard design (live re-optimisation)

Panel	Content
Controls (sidebar)	Risk aversion, max assets, ESG floor, turnover cap, noise level, mitigation on/off, solver choice
Live feed	Streaming prices (yfinance polling or replay of historical data at accelerated speed for reliable demos)
Optimisation	Button / timer triggers: update mu, Sigma, rebuild QUBO, warm-start QAOA, show convergence curve
Results	Weight bars per solver, risk / return / cost / ESG numbers, constraint status, cumulative PnL vs baselines
Hardware view	Qubit count, circuit depth, estimated logical resources for the current problem size

Use `st.session_state` to keep the previous solution, `st.cache_data` for downloads, and replay mode as the default (live feeds fail during demos). Keep the quantum problem small enough (10-14 qubits) to respond within a few seconds.

10.6 Day-by-day plan

Day	Learn (from this guide)	Build	Deliverable at end of day
1	Ch. 1, Ch. 2.1-2.3	Install stack, Bell circuit, small QAOA on 4-node MaxCut, download data	Working environment, notebook 01
2	Ch. 3, 9.1	Return / covariance estimation, QUBO builder with brute-force check, Gurobi + SA + equal-weight baselines, walk-forward backtest skeleton	QUBO verified against Gurobi; baseline backtest numbers
3	Ch. 2.4, Ch. 4	QAOA (ideal) on 10-14 qubit instances, noise model, multi-period version, first quantum backtest	Quantum vs classical on one test window
4	Ch. 5, 6, 7, 8	ZNE / DD / PEC ablation with mitiq, Stim surface-code p_L curves, resource table, RL agent (small), QAE demo	Ablation table, threshold plot, resource table
5	Ch. 9.3, 10.5	Crisis-period runs, statistical tests, Streamlit dashboard, write report	Report draft (15-20 pages) + dashboard

Time-saving advice
Do not try to master quantum mechanics. You only need: qubits, gates, measurement, expectation values, noise channels, stabilizer codes (conceptually). Spend the saved time on the classical side (data, baselines, statistics) because that is where evaluators check rigour. Start the long-running jobs (RL training, multi-seed noisy runs) at the end of Day 3 and let them run while you work on Day 4 theory.

10.7 Report outline (15-20 pages)

Section	Pages	Content
1 Introduction and problem statement	1	Motivation, contributions, honest scope
2 Background	2	Qubits, QAOA, QUBO, error models (shortened from this guide)
3 Formulation	2	Multi-period model, encoding, penalties, costs, ESG, dynamic constraints
4 Methods	3	QAOA/VQE design, noise, mitigation, logical-qubit modelling, baselines
5 Resource estimation	2	Stim results, fitted A and p_{th}, qubit and runtime table (item a)
6 Theory	2	Approximation ratio, noise bound, QAE sample complexity, TTS
7 Experiments	4	Ablation (item b), backtests incl. 2008 and 2020, significance tests (item c)
8 Dashboard	1	Architecture and screenshots (item d)
9 Limitations and conclusion	1	What is not proven, future work

11. Glossary, formula cheat sheet and reading list

11.1 Glossary

Term	Meaning
Qubit	Two-level quantum system, state alpha|0> + beta|1>
Shot	One run and measurement of a circuit
Hamiltonian H	Operator giving the energy; here it encodes the cost function
Ansatz	Parameterised circuit family used by a variational algorithm
Ground state	Lowest-energy eigenstate = optimal solution
Barren plateau	Region where gradients vanish exponentially, stalls training
Decoherence	Loss of quantum information to the environment
Fidelity	Overlap between ideal and actual state, 1 = perfect
Stabilizer	Pauli operator whose measurement reveals errors but not data
Syndrome	Pattern of stabilizer outcomes
Code distance d	Minimum number of errors causing an undetectable logical error
Threshold	Physical error rate below which bigger codes help
Lattice surgery	Logical two-qubit gates by merging surface-code patches
Magic state	Special resource state enabling non-Clifford (T) gates
Clifford circuit	Built from H, S, CNOT only; efficiently simulable classically (Stim)
NISQ	Noisy Intermediate-Scale Quantum era
CVaR	Conditional value at risk: mean loss in the worst (1-alpha) tail
TTS	Time to solution with 99% confidence

11.2 Formula cheat sheet

Topic	Formula
Born rule	P(x) = ||^2
Expectation	=
QUBO	min x^T Q x, x in {0,1}^n
Ising map	x = (1 - s)/2; J = Q_ij / 4
QAOA state	prod_k exp(-i beta_k H_M) exp(-i gamma_k H_C) |+>^n
Penalty	lambda (sum a_i x_i - c)^2
Circuit fidelity	F = exp(-epsilon G)
PEC overhead	gamma_total ~ exp(2 epsilon G), shots ~ gamma^2 / delta^2
Surface code qubits	2 d^2 - 1 per logical qubit
Logical error	p_L = A (p / p_th)^((d+1)/2)
QAE vs MC	O(1/eps) vs O(1/eps^2)
Grover	O(sqrt(2^n)) vs O(2^n)

11.3 Reading list (in the order to read them)

- **IBM Quantum Learning** (learning.quantum.ibm.com): free courses, 'Basics of quantum information' and 'Variational algorithm design'. Start here for Day 1.
- **Nielsen and Chuang**, Quantum Computation and Quantum Information (Chapters 1-2 only for the basics).
- **Farhi, Goldstone, Gutmann (2014)**, A Quantum Approximate Optimization Algorithm, arXiv:1411.4028.

- **Lucas (2014)**, Ising formulations of many NP problems, Frontiers in Physics (the formulation dictionary).
- **Brandhofer et al. (2022)**, Benchmarking the performance of portfolio optimization with QAOA, Quantum Machine Intelligence.
- **Egger et al. (2020)**, Quantum computing for finance: state of the art and future prospects, IEEE TQE. **Herman et al. (2023)**, A survey of quantum computing for finance.
- **Woerner and Egger (2019)**, Quantum risk analysis, npj Quantum Information (QAE for finance).
- **Fowler et al. (2012)**, Surface codes: towards practical large-scale quantum computation. **Gidney (2021)**, Stim: a fast stabilizer circuit simulator.
- **Temme, Bravyi, Gambetta (2017)**, Error mitigation for short-depth quantum circuits (ZNE and PEC). **LaRose et al. (2022)**, Mitiq.
- **Preskill (2018)**, Quantum computing in the NISQ era and beyond. **Babbush et al. (2021)**, Focus beyond quadratic speedups for error-corrected quantum advantage.
- **Gidney and Ekera (2021)**, resource-estimation methodology; **Azure Quantum Resource Estimator** documentation.
- **Rebentrost and Lloyd (2018)** and **Tang (2019)** on quantum portfolio algorithms and dequantization.
- **Ledoit and Wolf (2008)**, Robust performance hypothesis testing with the Sharpe ratio; **Bailey and Lopez de Prado (2014)**, Deflated Sharpe Ratio.

Please verify exact titles, venues and version-specific API details against the original sources before citing them in your report; this guide is a study aid, not a bibliography.